Week 04

리스트 · 튜플 · 집합 · 딕셔너리

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
def grade(score):
    if score < 0 or score > 100:    # 잘못된 입력 먼저
        return "오류"
    if score >= 90:                 # 엄격한 조건 먼저, >= 경계 포함
        return "A"
    ...
```

- 반복 패턴: 누적 · 개수 · 최댓값 · 모으기
- 검증: 경계 · 바로 아래 · 범위 끝 · 잘못된 입력

오늘의 목표

1. 네 가지 자료구조를 비교하고 **상황에 맞게** 고른다
2. 컴프리헨션, `sorted(key=...)`, 언패킹을 쓴다
3. 딕셔너리로 **개수 세기**와 **그룹 묶기**를 한다
4. 학생 성적 레코드를 **학과별로 요약**한다
5. AI 코드의 **반복 중 삭제 · 없는 키 · 원본 변경**을 찾아낸다

수업 흐름

구간	시간	활동
도입 · 개념	20분	자료구조 비교, 복사와 참조
안내 실습	45분	리스트 · 튜플 · 집합 · 딕셔너리
독립 · 짝 실습	35분	학생 성적 레코드 분석
공유 · 점검	20분	AI 코드 함정 3가지 공유

이번 주 Colab 실습 열기

네 가지 자료구조

	모양	순서	변경	중복	대표 용도
`list`	`[1, 2, 2]`	O	O	O	순서 있는 모음
`tuple`	`(1, 2)`	O	X	O	고정 레코드, 여러 값 반환
`set`	`{1, 2}`	X	O	X	중복 제거, 포함 여부
`dict`	`{"a": 1}`	O	O	키 X	이름으로 찾기, 집계

> "무엇을 자주 하는가?"로 고른다: 순서대로 훑기 · 중복 제거 · 이름으로 찾기

리스트 기본

```
scores = [72, 95, 88, 59]
scores.append(100)          # [72, 95, 88, 59, 100]
scores.insert(0, 67)        # [67, 72, 95, 88, 59, 100]
scores.pop()                # 100을 꺼내며 삭제
scores.remove(59)           # 값 59 삭제 (첫 번째 하나)
88 in scores                # True
scores.index(88)            # 3
sum(scores), min(scores), max(scores)
```

sort() vs sorted()

```
data = [3, 1, 2]
new = sorted(data, reverse=True)   # [3, 2, 1], data는 그대로
result = data.sort()               # data가 [1, 2, 3]으로 바뀜
print(result)                      # None !
```

	원본	반환값
`sorted(x)`	그대로	새 리스트
`x.sort()`	**바뀜**	`None`

`scores = scores.sort()` → scores가 **None**이 되는 흔한 실수

예상해 보기: 복사일까?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)        # ?
```

정답: `[1, 2, 3, 4]` — `b = a`는 **같은 리스트에 이름을 하나 더** 붙인 것

```
c = a.copy()    # 또는 list(a), a[:]
c.append(5)     # a는 그대로
```

리스트 컴프리헨션

`[식 for 변수 in 모음 if 조건]`

```
scores = [72, 95, 88, 59, 100, 67, 81]

passed = [s for s in scores if s >= 70]
curved = [min(s + 5, 100) for s in scores]
labels = ["합격" if s >= 70 else "재시험" for s in scores]
```

- 반복 + 조건 + 모으기를 한 줄로
- 집합 `{...}`, 딕셔너리 `{k: v for ...}`도 같은 문법

튜플과 언패킹

```
student = ("김민준", "컴퓨터공학과", 2)
name, dept, year = student          # 언패킹

x, y = y, x                         # 값 바꾸기

def min_max(values):
    return min(values), max(values)   # 튜플로 여러 값 반환

low, high = min_max([72, 95, 88])
student[2] = 3                      # TypeError: 바꿀 수 없음
```

집합: 중복 제거와 집합 연산

```
submitted = ["김민준", "이서연", "김민준", "박지호"]
len(set(submitted))                  # 3명

python_class = {"김민준", "이서연", "박지호", "최하은"}
data_class   = {"이서연", "최하은", "정도윤", "강지우"}
```

연산	결과
`python_class & data_class`	이서연, 최하은 (둘 다)
`python_class - data_class`	김민준, 박지호 (Python만)
`python_class | data_class`	6명 (하나라도)
`python_class ^ data_class`	4명 (한쪽만)

딕셔너리 기본

```
student = {"name": "김민준", "midterm": 78, "final": 85}

student["name"]                  # '김민준'
student["assignment"]            # KeyError!
student.get("assignment")        # None
student.get("assignment", 0)     # 0
student["final"] = 88            # 수정 / 추가

for key, value in student.items():
    print(key, value)
```

패턴 1: 개수 세기

```
answers = ["예", "아니오", "예", "예", "모름", "아니오", "예"]

counts = {}
for a in answers:
    counts[a] = counts.get(a, 0) + 1
# {'예': 4, '아니오': 2, '모름': 1}

from collections import Counter
Counter(answers).most_common(2)    # [('예', 4), ('아니오', 2)]
```

패턴 2: 그룹 묶기

```
pairs = [("컴퓨터공학과", 78), ("경영학과", 91),
         ("컴퓨터공학과", 85), ("간호학과", 88)]

groups = {}
for dept, score in pairs:
    groups.setdefault(dept, []).append(score)
# {'컴퓨터공학과': [78, 85], '경영학과': [91], '간호학과': [88]}

for dept, s_list in groups.items():
    print(dept, sum(s_list) / len(s_list))
```

sorted의 key

```
records = [("김민준", 85), ("이서연", 92), ("박지호", 85), ("최하은", 78)]

sorted(records, key=lambda r: r[1])                # 점수 오름차순
sorted(records, key=lambda r: r[1], reverse=True)  # 점수 내림차순
sorted(records, key=lambda r: (-r[1], r[0]))       # 점수↓, 같으면 이름↑
```

`(-r[1], r[0])` → 튜플은 **앞 원소부터** 비교
→ 점수가 같으면 이름으로

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- `sort` vs `sorted`, 복사와 참조, 컴프리헨션
- 집합 연산, 개수 세기, 그룹 묶기, 정렬 key

Level 2 바꾸기

- 80점 이상 이름만 모으기
- 학과 2개를 추가하고 **평균 높은 순**으로 출력

실습 3 (Level 3): 성적 레코드 분석

딕셔너리의 리스트 = 표 데이터의 기본 모양

```
students = [
    {"name": "김민준", "dept": "컴퓨터공학과", "midterm": 78, "final": 85, ...},
    {"name": "이서연", "dept": "정보보안학과", "midterm": 92, "final": 88, ...},
    ...
]
```

1. 총점 = 중간 40% + 기말 60% → **새 레코드**
2. 학과별 인원 · 평균 · 최고 득점자
3. 상위 3명 (동점 이름순), 출석 미달자 집합

학과별 요약 코드

```
by_dept = {}
for r in rows:
    by_dept.setdefault(r["dept"], []).append(r)

for dept, members in sorted(by_dept.items()):
    avg = sum(m["total"] for m in members) / len(members)
    best = max(members, key=lambda m: m["total"])
    print(dept, len(members), round(avg, 1), best["name"])

top3 = sorted(rows, key=lambda r: (-r["total"], r["name"]))[:3]
low_attendance = {r["name"] for r in rows if r["attendance"] < 12}
```

요약도 검증한다

```
# 빠진 학생 없음
assert sum(len(m) for m in by_dept.values()) == len(students)
# 1등이 진짜 최고점
assert top3[0]["total"] == max(r["total"] for r in rows)
# 원본은 그대로
assert students[0]["midterm"] == 78
```

> 요약 결과는 **원본과 대조**할 수 있어야 믿을 수 있다

AI 함정 1: 반복 중 삭제

```
scores = [55, 58, 90, 45, 70]
for s in scores:
    if s < 60:
        scores.remove(s)
print(scores)          # [58, 90, 70]  ← 58이 남았다!
```

55를 지우면 58이 앞으로 당겨져 **검사를 건너뜀**

```
kept = [s for s in scores if s >= 60]   # 남길 것만 모은 새 리스트
```

AI 함정 2 · 3

없는 키에 더하기

```
counts[a] += 1                          # KeyError
counts[a] = counts.get(a, 0) + 1        # OK
```

함수가 몰래 원본을 바꾸기

```
def add_bonus(score_list, bonus):
    for i in range(len(score_list)):
        score_list[i] += bonus          # 원본 수정!
    return score_list

def add_bonus_safe(score_list, bonus):
    return [s + bonus for s in score_list]
```

AI 코드 체크리스트

확인 질문	테스트 방법
반복 중 리스트를 지우나?	연속된 삭제 대상 `[55, 58, ...]`
없는 키를 바로 꺼내나?	처음 등장하는 키가 있는 입력
원본을 바꾸나?	실행 후 원본 `assert`
동점 처리는?	같은 점수 두 명

프롬프트: **"원본은 바꾸지 말고 새 리스트를 반환, 동점이면 이름 가나다순"**

이번 주 과제 (10점)

문제	내용	배점
1	`word_count(text)` 단어 빈도 (소문자, 문장부호 제거)	2
2	`compare_classes(a, b)` 집합으로 두 반 비교	2
3	`dept_summary(records)` 학과별 인원·평균·최고 (동점 처리)	3
4	`top_n`, `drop_below` – 원본을 지켜라 (AI 협업)	3

문제 4: AI의 첫 코드가 **어떤 확인에서 실패했는지** 기록

자기 점검

1. `b = a`와 `b = a.copy()`의 차이는?
2. `scores = scores.sort()` 후 scores의 값은?
3. 두 명단에서 "둘 다 듣는 학생"을 구하는 연산은?
4. 학과별로 점수를 모으는 딕셔너리 패턴은?
5. 반복 중 `remove`를 쓰면 왜 원소를 건너뛸까?

정리와 다음 주

오늘	핵심
리스트	`sorted` vs `sort()`, `.copy()`, 컴프리헨션
튜플 · 집합	언패킹, 중복 제거, `&` `|` `-`
딕셔너리	`get`, 개수 세기, `setdefault` 그룹
정렬	`key=lambda r: (-점수, 이름)`
AI 검증	반복 중 삭제 · 없는 키 · 원본 변경 · 동점

다음 주 · Week 05 함수 · 모듈 · 테스트 가능한 코드